

Manual de instalación: PostgreSQL 17 en Ubuntu Server 24.04 LTS

Alcance: PostgreSQL 17 (repositorio oficial PGDG) + PostGIS 3 + pg_cron + extensiones recomendadas + acceso remoto seguro + pgAdmin 4, sobre una VM con volumen dedicado montado en `/var/lib/postgresql`.

Convenciones: los comandos se ejecutan por SSH en la VM salvo que se indique lo contrario. Los valores entre `<...>` son marcadores que debes reemplazar (IPs, redes, contraseñas). Las versiones menores (17.x, 3.5.x) pueden variar levemente según la fecha de instalación; el procedimiento es el mismo.

Índice

1. Preparación del sistema
 2. Instalación de PostgreSQL 17 (repositorio PGDG)
 3. Verificación del clúster y del volumen dedicado
 4. Extensiones: PostGIS, pg_cron y contrib
 5. Configuración base del servidor (`postgresql.conf`)
 6. Acceso remoto seguro (listen_addresses, pg_hba, TLS, firewall)
 7. Roles y credenciales recomendadas
 8. pgAdmin 4
 9. Respaldos y mantenimiento
 10. Checklist final de verificación
 11. Consideraciones institucionales
-

1. Preparación del sistema

1.1 Actualizar el sistema operativo

```
sudo apt update && sudo apt upgrade -y
sudo reboot    # solo si se actualizó el kernel
```

1.2 Verificar el volumen dedicado

El disco para datos debe estar montado en `/var/lib/postgresql` **antes** de instalar, para que el clúster se cree directamente sobre él:

```
findmnt /var/lib/postgresql
df -h /var/lib/postgresql
```

`findmnt` debe mostrar un dispositivo (ej. `/dev/vdb1`). Crítico: confirma que el montaje **persiste tras reinicios**, es decir, que está en `/etc/fstab` idealmente por UUID:

```
grep -i postgres /etc/fstab
sudo blkid          # para obtener el UUID del dispositivo si falta la entrada
```

Si no está en `fstab`, agrégalo (ejemplo):

```
UUID=<uuid-del-volumen> /var/lib/postgresql ext4 defaults,nofail 0 2
```

y valida sin reiniciar con `sudo mount -a && findmnt /var/lib/postgresql`. Si el disco no estuviera en `fstab` y la VM se reinicia, PostgreSQL escribiría en el disco raíz sin que nadie lo note: es el error silencioso más caro de esta instalación.

1.3 Zona horaria y locale

```
sudo timedatectl set-timezone America/Santiago
timedatectl
```

Para ordenamiento alfabético correcto en español (tildes, ñ), genera el locale **antes** de crear el clúster:

```
sudo locale-gen es_CL.UTF-8
locale -a | grep es_CL
```

Si omites esto, el clúster se creará con el locale del sistema (usualmente `C.UTF-8` en imágenes cloud), que funciona pero ordena texto en español de forma no natural. En la sección 3.2 se indica cómo recrear el clúster si quedó con el locale equivocado (es barato mientras no haya datos).

2. Instalación de PostgreSQL 17 (repositorio PGDG)

Ubuntu 24.04 trae PostgreSQL 16 en sus repos; la 17 se instala desde el repositorio oficial del proyecto (PGDG), que además entrega actualizaciones menores más rápido.

```
sudo apt install -y postgresql-common ca-certificates
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh -y
sudo apt update
sudo apt install -y postgresql-17 postgresql-client-17
```

Notas:

- Los módulos *contrib* (pg_stat_statements, pgcrypto, pg_trgm, etc.) vienen incluidos en el paquete `postgresql-17`; no requieren paquete aparte.
- Al finalizar, el servicio queda creado e iniciado automáticamente y el clúster `17/main` se inicializa en `/var/lib/postgresql/17/main`.

Verificación inmediata:

```
pg_lsclusters
sudo systemctl status postgresql@17-main
sudo -u postgres psql -c "SELECT version();"
```

`pg_lsclusters` debe mostrar el clúster `17 main` con estado `online` y puerto `5432`.

3. Verificación del clúster y del volumen dedicado

3.1 Confirmar el data directory

```
sudo -u postgres psql -c "SHOW data_directory;"
```

Debe responder `/var/lib/postgresql/17/main`. Confirma que efectivamente vive sobre el volumen dedicado:

```
df -h /var/lib/postgresql/17/main
```

3.2 (Solo si aplica) Recrear el clúster con locale correcto

Si el clúster quedó con `C.UTF-8` y quieres `es_CL.UTF-8`, hazlo **ahora, antes de cargar datos**:

```
sudo pg_dropcluster 17 main --stop
sudo pg_createcluster 17 main --locale=es_CL.UTF-8 --start
sudo -u postgres psql -l      # verifica collate/ctype de las bases plantilla
```

4. Extensiones: PostGIS, pg_cron y contrib

4.1 PostGIS

```
sudo apt install -y postgresql-17-postgis-3 postgresql-17-postgis-3-scripts
sudo apt install -y postgis      # opcional: utilitarios CLI (shp2pgsql,
raster2pgsql)
```

PostGIS se habilita **por base de datos**. Ejemplo sobre una base de trabajo:

```
sudo -u postgres psql
```

```
CREATE DATABASE geodatos;
\c geodatos
CREATE EXTENSION postgis;
CREATE EXTENSION postgis_topology;    -- si usarás topología
SELECT PostGIS_Full_Version();
```

4.2 pg_cron

```
sudo apt install -y postgresql-17-cron
```

pg_cron requiere precargarse al inicio del servidor. Edita `/etc/postgresql/17/main/postgresql.conf` (o usa el archivo de la sección 5) y define:

```
shared_preload_libraries = 'pg_cron,pg_stat_statements'
cron.database_name = 'postgres'           # base donde vivirán los jobs
cron.timezone = 'America/Santiago'       # los horarios de cron en hora local
```

Reinicia y crea la extensión en esa base:

```
sudo systemctl restart postgresql@17-main
sudo -u postgres psql -c "CREATE EXTENSION pg_cron;"
```

Prueba con un job de mantenimiento nocturno:

```
SELECT cron.schedule('vacuum-nocturno', '0 3 * * *', 'VACUUM ANALYZE;');
SELECT * FROM cron.job;                -- jobs programados
SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 10; --
historial
```

Consideraciones de pg_cron:

- Ejecuta **SQL**, no comandos de shell. Los respaldos con `pg_dump` van en el cron del sistema operativo (sección 9), no aquí.
- Para correr un job en otra base usa `cron.schedule_in_database(...)`.
- Para desprogramar: `SELECT cron.unschedule('vacuum-nocturno');`

4.3 Extensiones contrib recomendadas

Se habilitan por base según necesidad (`CREATE EXTENSION <nombre>;`):

Extensión	Para qué sirve
<code>pg_stat_statements</code>	Estadísticas de consultas (ya precargada en 4.2); imprescindible para diagnosticar rendimiento
<code>pgcrypto</code>	Funciones criptográficas (hash, cifrado de columnas)
<code>pg_trgm</code>	Búsqueda difusa / similitud de texto, índices para <code>LIKE '%...%'</code>
<code>unaccent</code>	Búsquedas ignorando tildes (muy útil con datos en español)
<code>postgres_fdw</code>	Consultar otras bases PostgreSQL como tablas externas
<code>tablefunc</code>	Tablas cruzadas (<code>crosstab</code>)

Nota: para UUIDs ya no necesitas `uuid-oss`; usa la función nativa `gen_random_uuid()`.

5. Configuración base del servidor (`postgresql.conf`)

Archivo: `/etc/postgresql/17/main/postgresql.conf`. Ajusta memoria según la RAM real de la VM (`free -h`). Valores de referencia para una VM de **8 GB** dedicada a base de datos:

```
# --- Conexiones ---
listen_addresses = '*'           # ver sección 6: el control real está en pg_hba +
firewall
max_connections = 100

# --- Memoria ---
shared_buffers = 2GB             # ~25 % de la RAM
effective_cache_size = 6GB       # ~75 % de la RAM (estimación, no reserva)
work_mem = 32MB                  # por operación de ordenamiento/hash; subir con
cautela
maintenance_work_mem = 512MB     # VACUUM, CREATE INDEX

# --- WAL / checkpoints ---
max_wal_size = 2GB
min_wal_size = 512MB
summarize_wal = on               # habilita respaldos incrementales (novedad de la
17)

# --- Precarga (sección 4.2) ---
shared_preload_libraries = 'pg_cron,pg_stat_statements'
cron.database_name = 'postgres'
cron.timezone = 'America/Santiago'

# --- Zona horaria y logs ---
timezone = 'America/Santiago'
log_timezone = 'America/Santiago'
```

```

log_line_prefix = '%m [%p] %u@%d %r '
log_min_duration_statement = 1000 # registra consultas > 1 s
log_connections = on              # trazabilidad de accesos (contexto
institucional)
log_disconnections = on

# --- Seguridad ---
password_encryption = scram-sha-256 # default en 17; se explicita por claridad
ssl = on                          # Ubuntu lo deja activo con certificado
"snakeoil"

```

Aplica con:

```
sudo systemctl restart postgresql@17-main
```

(**restart** es necesario por **shared_preload_libraries**; el resto aceptaría **reload**.)

6. Acceso remoto seguro

Objetivo: que el servidor sea alcanzable desde las estaciones/servidores que corresponda, sin quedar expuesto indiscriminadamente. La defensa tiene tres capas: **pg_hba.conf** (quién y cómo se autentica), TLS (cifrado en tránsito) y firewall (qué IPs llegan al puerto).

6.1 pg_hba.conf

Archivo: **/etc/postgresql/17/main/pg_hba.conf**. Las reglas se evalúan en orden, gana la primera coincidencia. Configuración recomendada:

#	TYPE	DATABASE	USER	ADDRESS	METHOD
# Administración local (usuario de sistema postgres, sin contraseña)					
local	all	postgres			peer
# Conexiones locales de otros roles					
local	all	all			scram-sha-256
host	all	all	127.0.0.1/32		scram-sha-256
host	all	all	:::1/128		scram-sha-256
# Acceso remoto SOLO desde redes autorizadas, cifrado y con SCRAM					
hostssl	all	all	<RED_INTERNA>/24		scram-sha-256
hostssl	all	all	<IP_ESTACION>/32		scram-sha-256
# El superusuario postgres NUNCA entra por red					
host	all	postgres	0.0.0.0/0		reject

Reglas de oro:

- `hostssl` (no `host`) para todo lo remoto: obliga TLS.
- Jamás uses `trust` fuera de un laboratorio desechable, ni `md5` (obsoleto frente a `scram-sha-256`).
- Si realmente necesitas abrir a cualquier origen (`0.0.0.0/0`), hazlo consciente de que el puerto 5432 expuesto a internet recibe escaneos y ataques de diccionario en minutos. En ese caso son obligatorios: contraseñas largas generadas (sección 7), `hostssl`, firewall del proveedor cloud restringiendo orígenes, y de preferencia VPN. Para un servidor institucional, lo correcto es autorizar rangos específicos.

Aplica cambios de `pg_hba` con recarga (no requiere reinicio):

```
sudo systemctl reload postgresql@17-main
```

6.2 TLS con certificado propio

Ubuntu deja `ssl = on` con un certificado autofirmado genérico ("snakeoil"). Funciona, pero conviene generar uno propio con el nombre del servidor (o usar un certificado de la CA institucional si existe):

```
sudo openssl req -new -x509 -days 3650 -nodes -text \
  -out /etc/postgresql/17/main/server.crt \
  -keyout /etc/postgresql/17/main/server.key \
  -subj "/CN=<hostname-o-ip-del-servidor>"
sudo chown postgres:postgres /etc/postgresql/17/main/server.{crt,key}
sudo chmod 600 /etc/postgresql/17/main/server.key
```

En `postgresql.conf`:

```
ssl_cert_file = '/etc/postgresql/17/main/server.crt'
ssl_key_file  = '/etc/postgresql/17/main/server.key'
```

y `sudo systemctl restart postgresql@17-main`.

6.3 Firewall (UFW)

```
sudo ufw allow OpenSSH                                # ¡PRIMERO! o pierdes el
SSH
sudo ufw allow from <RED_INTERNA>/24 to any port 5432 proto tcp
sudo ufw allow from <IP_ESTACION>/32 to any port 5432 proto tcp
sudo ufw enable
sudo ufw status numbered
```

Si la VM está en un proveedor cloud (p. ej. las contratadas vía Hosting.cl), replica estas restricciones también en el firewall/grupo de seguridad del panel del proveedor: dos capas independientes.

6.4 Verificación del acceso remoto

En el servidor:

```
ss -tlnp | grep 5432      # debe escuchar en 0.0.0.0:5432
```

Desde tu estación de trabajo:

```
psql "host=<IP_SERVIDOR> port=5432 dbname=postgres user=<rol> sslmode=require"
```

Dentro de la sesión, `SELECT ssl FROM pg_stat_ssl WHERE pid = pg_backend_pid();` debe devolver `t`.

7. Roles y credenciales recomendadas

Principios: el superusuario `postgres` se usa solo localmente y para administración excepcional; el trabajo diario y las aplicaciones usan roles propios con privilegios mínimos; toda contraseña es única, generada, de al menos 20 caracteres, y vive en un gestor de contraseñas institucional, nunca en correos ni scripts en texto plano.

7.1 Generar contraseñas

```
openssl rand -base64 24
```

Genera una distinta por rol. (No adoptes contraseñas "de ejemplo" de ningún manual, incluido este.)

7.2 Estructura de roles sugerida

```
-- 1) Contraseña al superusuario (aunque solo entre por 'local peer', por higiene)
ALTER USER postgres PASSWORD '<generada>';

-- 2) Administrador operativo (tu cuenta de trabajo diario, sin ser superusuario)
CREATE ROLE admin_uia LOGIN CREATEDB CREATEROLE PASSWORD '<generada>';

-- 3) Rol de aplicación (uno por sistema/servicio, con límite de conexiones)
CREATE ROLE app_servicio LOGIN PASSWORD '<generada>' CONNECTION LIMIT 20;

-- 4) Rol de solo lectura (para análisis y reportería)
CREATE ROLE lectura NOLOGIN;
CREATE ROLE analista LOGIN PASSWORD '<generada>' IN ROLE lectura;

-- 5) Base de trabajo con dueño explícito
CREATE DATABASE sistema_x OWNER app_servicio;

-- 6) Permisos de solo lectura sobre esa base (ejecutar conectado a sistema_x)
GRANT CONNECT ON DATABASE sistema_x TO lectura;
GRANT USAGE ON SCHEMA public TO lectura;
```



```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lectura;
ALTER DEFAULT PRIVILEGES FOR ROLE app_servicio IN SCHEMA public
GRANT SELECT ON TABLES TO lectura;
```

Notas:

- Desde PostgreSQL 15, el esquema `public` ya no permite `CREATE` a cualquier usuario; los objetos los crea el dueño de la base o quien reciba el permiso explícito. No hay que "cerrar" nada extra.
- Revisa roles y atributos cuando quieras con `\du` en `psql`.

7.3 Credenciales en clientes sin exponerlas

Para scripts y conexiones frecuentes usa `~/.pgpass` (en tu estación), nunca la contraseña en el comando:

```
# formato: host:puerto:base:usuario:contraseña
<IP_SERVIDOR>:5432:*:admin_uia:<contraseña>
```

```
chmod 600 ~/.pgpass
```

8. pgAdmin 4

Dos modalidades; elige según el uso:

- **Escritorio en tu estación de trabajo (recomendada si los usuarios son 1–3 personas):** cero superficie adicional expuesta en el servidor; solo necesitas el puerto 5432 ya abierto. En Windows se instala desde el sitio oficial de pgAdmin.
- **Modo web en el servidor (si el equipo necesita un acceso compartido por navegador):** instala en la VM lo siguiente.

8.1 pgAdmin 4 en modo web (en el servidor)

```
curl -fsS https://www.pgadmin.org/static/packages_pgadmin_org.pub \
| sudo gpg --dearmor -o /usr/share/keyrings/packages-pgadmin-org.gpg

sudo sh -c 'echo "deb [signed-by=/usr/share/keyrings/packages-pgadmin-org.gpg]
https://ftp.postgresql.org/pub/pgadmin/pgadmin4/apt/$(lsb_release -cs) pgadmin4
main" > /etc/apt/sources.list.d/pgadmin4.list'

sudo apt update
sudo apt install -y pgadmin4-web
sudo /usr/pgadmin4/bin/setup-web.sh # define aquí el correo y contraseña del
usuario inicial
```

Queda servido por Apache en `http://<IP_SERVIDOR>/pgadmin4`. Consideraciones:

- Abre el puerto en UFW **solo** a las redes autorizadas: `sudo ufw allow from <RED_INTERNA>/24 to any port 80,443 proto tcp`.
- Sirviéndolo por red, corresponde HTTPS: habilita `sudo a2enmod ssl && sudo a2ensite default-ssl && sudo systemctl reload apache2` con el certificado autofirmado, o idealmente un certificado institucional.
- La cuenta inicial de pgAdmin (correo + contraseña de `setup-web.sh`) es independiente de los roles de PostgreSQL: también debe ser generada y guardada en el gestor.

8.2 Registrar el servidor en pgAdmin

En pgAdmin: *Add New Server* → pestaña *Connection*: Host `<IP_SERVIDOR>` (o `localhost` si es modo web en la misma VM), Port `5432`, Maintenance DB `postgres`, Username `admin_uia`; pestaña *Parameters/SSL*: SSL mode `Require`.

9. RespalDOS y mantenimiento

9.1 Respaldo lógico diario (cron del sistema)

Crea `/usr/local/bin/backup_pg.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
DEST=/var/backups/postgresql
FECHA=$(date +%F)
mkdir -p "$DEST"
for DB in $(psql -At -c "SELECT datname FROM pg_database WHERE NOT datistemplate
AND datname <> 'postgres';"); do
    pg_dump -Fc "$DB" > "$DEST/${DB}_${FECHA}.dump"
done
# Retención: 14 días
find "$DEST" -name '*.dump' -mtime +14 -delete
```

```
sudo chmod +x /usr/local/bin/backup_pg.sh
sudo tee /etc/cron.d/backup-postgresql > /dev/null << 'EOF'
30 2 * * * postgres /usr/local/bin/backup_pg.sh
EOF
```

Importante: un respaldo que vive solo dentro de la misma VM no es un respaldo. Copia periódicamente `/var/backups/postgresql` a otra máquina o almacenamiento (rsync/scp hacia otro servidor, o el mecanismo de snapshots del proveedor, idealmente ambos). Restaura de prueba al menos una vez: `pg_restore -d <base_destino> archivo.dump`.

9.2 Respaldo físico e incremental (opcional, novedad de la 17)

Con `summarize_wal = on` (sección 5) puedes tomar una base completa y luego incrementales, combinables con `pg_combinebackup`:

```
pg_basebackup -D /var/backups/base_full -c fast
pg_basebackup -D /var/backups/base_incr --
incremental=/var/backups/base_full/backup_manifest
```

Útil cuando las bases crezcan; para partir, el respaldo lógico diario de 9.1 es suficiente.

9.3 Mantenimiento

- **autovacuum** viene activo por defecto: no lo desactives.
- El job de pg_cron de la sección 4.2 (**VACUUM ANALYZE** nocturno) complementa, no reemplaza, al autovacuum.
- Actualizaciones menores: `sudo apt update && sudo apt upgrade` las instala desde PGDG; implican reinicio breve del servicio, agenda una ventana.
- Rendimiento: revisa periódicamente `SELECT query, calls, mean_exec_time FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 15;`

10. Checklist final de verificación

Ejecuta en orden; todo debe cumplirse antes de dar el servidor por operativo:

1. `pg_lsclusters` → clúster **17 main**, estado **online**, puerto 5432.
2. `sudo -u postgres psql -c "SHOW data_directory;"` → `/var/lib/postgresql/17/main`, y `findmnt /var/lib/postgresql` muestra el volumen dedicado con entrada en `/etc/fstab`.
3. `SELECT version();` → PostgreSQL 17.x.
4. En la base de trabajo, `\dx` lista **postgis** (y `SELECT PostGIS_Full_Version();` responde).
5. `SELECT * FROM cron.job;` muestra el job programado y, al día siguiente, `cron.job_run_details` registra ejecución exitosa.
6. Desde tu estación: `psql "host=<IP_SERVIDOR> sslmode=require ..."` conecta, y `pg_stat_ssl` confirma TLS.
7. Desde una IP **no autorizada** (si puedes probarlo): la conexión es rechazada.
8. `sudo ufw status` muestra solo SSH y 5432 (y 80/443 si hay pgAdmin web) restringidos a los orígenes correctos; el firewall del proveedor replica las reglas.
9. `\du` muestra los roles creados; nadie salvo **postgres** es superusuario; **postgres** no puede conectar por red.
10. El script de respaldo corrió (archivo `.dump` del día presente) y una restauración de prueba funcionó.
11. Credenciales de todos los roles y de pgAdmin registradas en el gestor de contraseñas institucional.

11. Consideraciones institucionales

- **Protección de datos (Ley 21.719 y 19.628):** si el servidor almacenará datos personales, el cifrado en tránsito (TLS), el control de acceso por roles y el registro de conexiones (`log_connections`) configurados en este manual son la línea base técnica; documenta además la finalidad y responsables según el procedimiento interno.
- **Inventario:** registra la VM, IP, puerto, versión, bases, extensiones y responsables en el inventario de activos de información; este manual puede adjuntarse como evidencia de configuración.

- **Exposición mínima:** aunque el requerimiento sea "accesible por IPs externas", externo debe significar *rangos identificados* (oficinas, otros servidores, VPN institucional), no internet completo. Amplía el `pg_hba.conf` y el firewall a medida que aparezcan necesidades reales, no de forma preventiva.
- **Trazabilidad de cambios:** los archivos `postgresql.conf` y `pg_hba.conf` conviene versionarlos (aunque sea una copia fechada antes de cada cambio: `sudo cp pg_hba.conf pg_hba.conf.$(date +%F)`)).